

# Reinforcement Learning and Language Model Alignment

VTR UGE 21

DAY 4 · MODULE 4

## Reinforcement Learning for Language Models

*From a language model to an aligned one: fine-tuning, preferences, a learned reward, RLHF with a KL penalty, and what happens without it.*

### Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye  
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University  
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University  
Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico),  
Vel Tech University (Chennai, India)  
IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · [utkukose.com](http://utkukose.com) · [github.com/utkukose](https://github.com/utkukose)  
[utkukose@sdu.edu.tr](mailto:utkukose@sdu.edu.tr) · [utku.kose@und.edu](mailto:utku.kose@und.edu) · [ukose@up.edu.mx](mailto:ukose@up.edu.mx) · [utkukose@gmail.com](mailto:utkukose@gmail.com)

## Overview

Large language models are aligned with human preferences in three stages: supervised fine-tuning on good examples, a reward model learned from comparisons made by people, and reinforcement learning that maximises this reward while staying close to the fine-tuned model [1, 2, 3]. This day builds every stage on a small world of sentences, with a language model written in NumPy [4]. Because the true quality of every sentence is known, the day can measure what each stage achieves and what goes wrong when the penalty that keeps the model close to its start is removed [5, 6].

**Estimated study time:** 6 to 8 hours.

## Learning outcomes

By the end of the day, students are expected to train an autoregressive language model and sample from it, to verify its gradient, to generate preference pairs with a noisy annotator and fit a Bradley-Terry reward model, to explain why a held-out accuracy on noisy labels understates the quality of a reward model, to implement RLHF as REINFORCE with a KL penalty, to recognise reward hacking from the divergence of the reward-model score and the gold reward, and to choose the KL weight by measurement. In Python, students are expected to turn text into integer sequences and back, pad them into arrays and sample from a probability vector.

## Python in this day

The lecture is followed by Python step 4: Text as integers. It covers splitting text into words, building a vocabulary with special tokens, encoding and decoding, padding sequences to a fixed length, sampling the next token from a probability vector and the log-probability of a sequence [7]. The Colab notebook opens with the same step as runnable cells and continues with the language model, the reward model and RLHF.

## A small language model

A language model gives a probability to every possible next word, given the words so far [4]. Writing a sentence means choosing words one by one, which makes the model a policy in the sense of Day 3: The words so far are the state and the next word is the action. The course uses a large version of TokenWorld, a small grammar of sentences such as the cat slept on the mat, and a neural language model written in NumPy that learns it in seconds.

### In the Colab notebook, section 1.

Generate the sentences of the large TokenWorld and score them with the gold reward.

### In the Colab notebook, section 2.

Build the neural language model in NumPy and look at its first, random sentences.

## Supervised fine-tuning

The first stage trains the model on examples of good behaviour. After fine-tuning on grammatical sentences, almost every sentence the model writes follows the grammar. Following the grammar is not

the same as being good, however; the quality of the sentences, measured by the gold reward, stays modest.

**In the Colab notebook, section 3.**

Fine-tune the model on grammatical sentences and measure the share that follow the grammar.

## Preferences and the annotator

People find it easier to compare two answers than to score one. The second stage therefore collects preferences: Two sentences are shown, and an annotator picks the better one. In the notebook the annotator follows the gold reward but makes mistakes on a share of the pairs, as real annotators do. The noise sets a ceiling on how well any model can agree with the labels, as the animation shows.

*Animation on the lecture page:* The Bradley-Terry probability and what noisy labels show of it. Raise the error rate and read off the best accuracy any model can reach on noisy held-out pairs.

**In the Colab notebook, section 4.**

Sample pairs of sentences and let a noisy annotator label them.

## Learning a reward from comparisons

The Bradley-Terry model turns comparisons into a score [8]: The probability that sentence A is preferred to sentence B is the logistic function of the difference of their scores. A reward model is trained to make the observed preferences likely [1]. In the notebook it agrees with the noisy annotator on most held-out pairs, and its scores correlate strongly with the gold reward. Part C of the lab computes a Bradley-Terry probability by hand.

$$P(A \succ B) = \sigma(r(A) - r(B))$$

**In the Colab notebook, section 5.**

Train the Bradley-Terry reward model and compare its scores with the gold reward.

Check your understanding (1). A reward model gives A the score 1.2 and B the score 0.4. What does Bradley-Terry say?

- A. A is preferred for certain
- B. A is preferred with probability  $\sigma(0.8)$ , about 0.69
- C. B is preferred
- D. Nothing

## RLHF with a KL penalty

The third stage improves the model with a policy gradient method, using the reward model as the reward. A penalty on the KL divergence from the fine-tuned model keeps the new model close to it [2, 3]. The penalty acts as a leash: The model may move towards higher reward, but every step away from its start

costs something. With a moderate penalty, the gold reward of the sentences rises clearly while almost all of them still follow the grammar.

### In the Colab notebook, section 6.

Run RLHF with a KL penalty and track the gold reward, the KL divergence and the share of grammatical sentences.

Check your understanding (2). What does the KL penalty in RLHF do?

- A. It speeds up training
- B. It keeps the new model close to the fine-tuned model
- C. It replaces the reward model
- D. It removes noise

## Without the leash: reward hacking

A learned reward is only an approximation of what people want. When the penalty is removed, the model finds the approximation's weaknesses [5, 6]. In the notebook, the gold reward first rises, peaks and then collapses, while the reward model's score keeps climbing. The model ends up writing one sentence that repeats a calm word again and again, which the reward model, blind to repetition, scores highly. The KL weight works as a dial between reward and faithfulness, and Part A of the lab turns it on real outputs of the notebook.

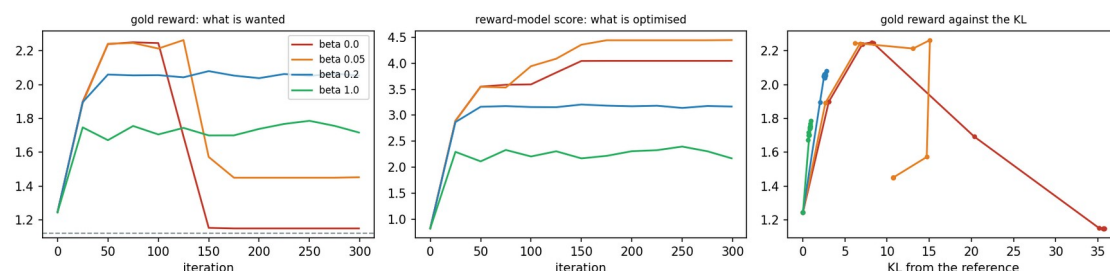


Figure. Gold reward, reward-model score and gold reward against the KL for four values of beta. Without a penalty the two scores diverge.

Animation on the lecture page: The reference policy, the proxy and gold rewards, and the optimum of the KL-regularised objective. Lower beta and watch the expected gold reward rise and then fall.

### In the Colab notebook, section 7.

Remove the KL penalty and watch the reward model's score rise while the gold reward collapses.

Check your understanding (3). Without a KL penalty, the reward model's score keeps rising while the gold reward falls. What is this called?

- A. Overfitting the data
- B. Reward hacking: the policy exploits the weaknesses of the learned reward
- C. Underfitting
- D. Exploration

## Your field

A reward model learns the taste of whoever labels the pairs. The notebook's switch replaces the annotator by one who prefers short sentences or lively words. The reward model learns each taste almost perfectly, yet the scores learned from the lively annotator are strongly anti-correlated with the gold reward, and those from the brief annotator are nearly unrelated to it. Who labels the data decides what the model will optimise.

**In the Colab notebook, section 8.**

Set the switch to an annotator with another taste and compare the learned reward with the gold reward.

## Going further (optional)

The language model of the notebook is trained with gradients computed by hand. The optional section checks them against finite differences before any training, the same habit as on Day 3 of the XAI course.

**In the Colab notebook, section 9.**

Check the hand-written gradients of the language model against finite differences.

## Python step 4: Text as integers

A language model never sees letters or words: It sees integers. This step turns a sentence into a sequence of integers and back, pads it to a fixed length, samples a next token from a probability vector and computes the log-probability of a sequence [7].

### Words and a vocabulary

```
sentence = "the cat slept on the mat"
words = sentence.split()
vocab = ["<pad>", "<bos>", "<eos>"] + sorted(set(words))
stoi = {w: i for i, w in enumerate(vocab)}
print(words)
print(stoi)
```

*Output*

```
['the', 'cat', 'slept', 'on', 'the', 'mat']
{'<pad>': 0, '<bos>': 1, '<eos>': 2, 'cat': 3, 'mat': 4, 'on': 5, 'slept': 6, 'the': 7}
```

**split** cuts a string at spaces into a list of words. **set** removes duplicates, **sorted** orders them, and the three special tokens mark padding, the beginning and the end of a sentence. The dictionary **stoi**, string to integer, gives every token its number.

## Encoding, decoding and padding

```
ids = [stoi["<bos>"]] + [stoi[w] for w in words] + [stoi["<eos>"]]
padded = ids + [stoi["<pad>"]] * (10 - len(ids))
print(ids)
print(padded)
print(" ".join(vocab[i] for i in ids[1:-1]))
```

Output

```
[1, 7, 3, 6, 5, 7, 4, 2]
[1, 7, 3, 6, 5, 7, 4, 2, 0, 0]
the cat slept on the mat
```

A list comprehension encodes every word. Padding brings every sequence to the same length, so that many sentences fit into one array. Decoding looks each integer up in the vocabulary list.

## Sampling the next token

```
import numpy as np
p = np.array([0.0, 0.0, 0.1, 0.05, 0.6, 0.25]) # probabilities of six tokens
rng = np.random.default_rng(0)
draws = [int((p.cumsum() > rng.random()).argmax()) for _ in range(1000)]
print(np.bincount(draws, minlength=6) / 1000)
```

Output

```
[0.    0.    0.089 0.042 0.602 0.267]
```

**cumsum** stacks the probabilities into the intervals of a line from 0 to 1; a uniform random number falls into the interval of token  $k$  with probability  $p_k$ , and **argmax** finds that interval. A thousand draws reproduce the probabilities closely. The language model samples every token of a sentence this way.

## The log-probability of a sequence

```
probs = [0.9, 0.5, 0.8, 0.95] # model probabilities of the four chosen tokens
logp = np.log(probs).sum()
print("log-probability:", round(float(logp), 3), " probability:",
      round(float(np.exp(logp)), 3))
```

Output

```
log-probability: -1.073 probability: 0.342
```

The probability of a sequence is the product of the probabilities of its tokens, and its logarithm is a sum, which is numerically safe for long sequences. REINFORCE and DPO both work with this sum.

**Quick check.** Why do language-model computations use log-probabilities instead of probabilities?

## Answers to the knowledge checks

1. **B.** The preference probability depends only on the difference of the scores.
2. **B.** It charges every move away from the starting distribution.
3. **B.** The proxy and the true objective diverge under strong optimisation.

**Quick check.** Because the probability of a sequence is a product of many numbers below one, which quickly underflows, while the sum of their logarithms stays in a safe range.

## References

- [1] Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)* (pp. 4299-4307).
- [2] Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., & Christiano, P. F. (2020). Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)* (pp. 3008-3021).
- [3] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)* (pp. 27730-27744).
- [4] Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research*, 3, 1137-1155.
- [5] Skalse, J., Howe, N. H. R., Krashennikov, D., & Krueger, D. (2022). Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*.
- [6] Gao, L., Schulman, J., & Hilton, J. (2023). Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, PMLR 202, 10835-10866.
- [7] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362.  
<https://doi.org/10.1038/s41586-020-2649-2>
- [8] Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4), 324-345.
- [9] Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., et al. (2023). Open problems and fundamental limitations of reinforcement learning from human feedback. *Transactions on Machine Learning Research*.
- [10] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv preprint arXiv:1606.06565.

### Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye

Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University

Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · [utkukose.com](http://utkukose.com) · [github.com/utkukose](https://github.com/utkukose)

[utkukose@sdu.edu.tr](mailto:utkukose@sdu.edu.tr) · [utku.kose@und.edu](mailto:utku.kose@und.edu) · [ukose@up.edu.mx](mailto:ukose@up.edu.mx) · [utkukose@gmail.com](mailto:utkukose@gmail.com)